

The Fall of Shurima

SENG201 Project Report - Team 76

Mohammed Shoeb Daniyal (moh28) | Xinyi Zhang (xzh250)

1. Application Overview and Structure

The Fall of Shurima is a JavaFX strategy/RPG game where the player chooses Aatrox or Xolaani as the main character, builds a guild, makes quest decisions, manages loyalty and madness, and fights bosses through a memory-game combat system.

Layer	Lines of Code	Classes / Files	Responsibility
Models	4608	Adventurer, Boss, BossAbility, BossFight, Character, Difficulty, Faction, Game, Guild, Item, ItemType, MemoryGame, Quest, QuestStoryChoices, QuestStoryEvents, StoryDrivenQuests	Contain core rules: • party state, • gold, potions, quest progression, • boss abilities, combat, win/loss state, • death and abandonment.
Controllers	5179	BossFightController, CharacterSelectController, DiceRollController, EndController, Quest1-6 Controllers, QuestChoiceController, ScreenUtil, SetupController, Shop Controller, StartController	• Connect FXML screens to model methods, • update labels/images/health bars, • handle button clicks/navigation.
Resource Files	2708	FXML, CSS	• Define screen layouts, styling, backgrounds, character sprites, boss images, and final result images.
Image Files	90	.png files	• Images, backgrounds, logos, symbols, characters

2. Key Design Decisions

- The application was implemented using a model-focused structure. The controllers mainly handle JavaFX screen changes, button actions, labels, images, and popups, while the game rules are stored in the model classes.
- A shared battle implementation was used instead of writing a separate combat system for every quest. All boss fights use BossFightController for the screen and BossFight for the battle rules. Different enemy behaviours are handled through the BossAbility enum.

- The quest system was implemented using reusable story-event classes. QuestStoryEvent stores the title, text, background image, and choices for a quest event. QuestStoryChoice stores the player's available decisions and their effects. QuestChoiceController then displays these events for multiple quests. This meant that Quest 2 to Quest 6 could share the same event-screen logic while still having different story content.
- Navigation between screens was implemented using the GameDataReceiver interface. Controllers that need the current Game object implement this interface. This replaced reflection-based method calls and made the code safer because the compiler can check that the controller has the correct setGameData(Game game) method.

3. Use of Inheritance, Interfaces

- A major inheritance relationship is Character -> Adventurer and Character -> Boss. Character stores shared behavior such as name, max health, current health, attack, defense, health clamping, resetting health, and death checks. Adventurer extends this with player-specific properties while boss extends it with enemy-specific properties such as gold drops, loyalty effects, lore description, special ability, and ability frequency.
- The second major inheritance structure is the quest hierarchy. Quest is an abstract superclass for all the quests. This lets all quests share common behavior such as quest ID, quest name, unlocking, completion, madness effects, and boss-fight lists. Each quest then implements its own boss fights, lore, and event outcomes. This made the quest system easier to extend.
- Interfaces were used to separate shared behaviors without forcing every class into the same inheritance tree. GameDataReceiver is implemented by controllers that need to receive the active Game object during screen navigation. StoryDrivenQuest is implemented by quests that provide lore screens and story events. These interfaces helped make the program easier to organize because controllers and quest screens can work with a common type instead of checking every class individually.
- Enums were also used to make important rule categories clear and less error prone. Using Enums for Difficulty, Faction, BossAbility made the rules more readable than using strings or raw numbers.

4. Unit Testing and Manual Testing

Unit tests focus on model logic because the model layer contains the game rules and can be tested without opening JavaFX windows.

5. What Went Well

- The model structure became clearer as the project developed.
- The shared boss-fight controller made it possible to support several unique bosses without duplicating whole battle screens.
- The story and theme became stronger once quest lore screens and final result text were added. The final screen now reflects the full journey by including survivors, dead characters, and abandoned characters.

6. Challenges and Improvements

- The largest challenge was balancing JavaFX screen flow with persistent game state. Returning to the map, restarting quests, and keeping choices remembered required careful handling of the Game and Quest objects.
- Combat balancing also changed many times. Boss damage, party size, potion use, and special abilities needed repeated testing, so the game felt challenging without soft-locking or ending too quickly.

- If we had more time, we would add save/load support, stronger GUI test automation, more polished animation, and a clearer asset manager for images and fallback backgrounds.

7. Effort and Contribution

Team member	Main responsibilities	Estimated hours	Contribution
Mohammed Shoeb Daniyal (moh28)	Game concept, story direction, quest design, testing, gameplay balancing, JavaFX integration support, report/README preparation.	Approx. 124 – 136 hours	50%
Xinyi Zhang (xzh250)	Model implementation, controller implementation, FXML/CSS support, unit testing, debugging, image/resource integration	Approx. 126 – 134 hours	50%

8. Conclusion

Overall, the project demonstrates an interactive JavaFX game with meaningful player choices, model-driven combat, multiple quest paths, permanent consequences, and a complete win/loss flow. The main learning outcome was understanding how to separate JavaFX interface code from reusable game logic while keeping the project testable and maintainable.

9. Declaration for the use of GenAI:

Generative AI tools were used as a support tool during this project. The tools were used to assist with:

- generating repetitive code such as getters and setters,
- improving the wording of the report and README,
- better understanding parts of the assignment requirements,
- generating image prompts and visual assets and receiving suggestions for debugging and testing.

All final code, design decisions, UML diagrams, report content, and submitted files were reviewed, edited, and approved by the team. The team takes full responsibility for the final implementation, testing, design choices, and submitted project work. AI-generated suggestions were not submitted without review or modification.

9. Riot Games Disclaimer:

This is a non-commercial student project created for SENG201 coursework. The project is inspired by Riot Games' League of Legends universe. Riot Games do not endorse, sponsor, or approve this project. Riot Games, League of Legends, Runeterra, and related characters are trademarks or intellectual property of Riot Games. This project is for educational use only and is not intended for sale or public commercial release.